

Lab 08 Practice: Neural Networks

CS201L: Artificial Intelligence Laboratory
Indian Institute of Technology, Dharwad

05 - Mar - 2026

Problem Statement

Using the Date Fruit dataset (`Date_Fruit_Datasets.csv`), implement and compare the performance of given architectures of Neural Networks. You are required to categorize the fruit samples into their respective classes based on image-extracted numerical features.

Dataset Properties

- **Total Samples:** 898
- **Total Features:** 34 numerical features
- **Target Attribute:** Class (multiclass)
- **Data Splits:** Use the `DateFruit_Train.csv` (60%), `DateFruit_Validation.csv` (20%), and `DateFruit_Test.csv` (20%).

Write a Python program to complete the following tasks.

Part A: Single and Two hidden layer Neural network architecture on the Date Fruit Dataset

Perform neural network classification and performance metric evaluation as listed below.

1. Load train, validation, and test data from original data: '`DateFruit_Train.csv`', '`DateFruit_Validation.csv`', '`DateFruit_Test.csv`', respectively.
2. Implement a neural network for classification using PyTorch with the following specifications:

- Input layer: 34 linear neurons (corresponding to real-valued input features).
 - All neurons in the hidden layers use the tanh activation function.
 - Hidden layers: Experiments with different numbers of hidden layers and different numbers of neurons in each hidden layer:
 - (a) Single hidden layer architectures:
 - Architecture 1: Input layer $\rightarrow$ hidden layer (34 neurons) $\rightarrow$ Output layer
 - Architecture 2: Input layer $\rightarrow$ hidden layer (64 neurons) $\rightarrow$ Output layer
 - (b) Two hidden layer architectures:
 - Architecture 1: Input layer $\rightarrow$ hidden layer (34 neurons) $\rightarrow$ hidden layer (34 neurons) $\rightarrow$ Output layer
 - Architecture 2: Input layer $\rightarrow$ hidden layer (64 neurons) $\rightarrow$ hidden layer (16 neurons) $\rightarrow$ Output layer
 - Output layer: 7 neurons (*Softmax* activation function).
3. Train the neural network using:
 - Optimizer: Stochastic Gradient Descent (SGD)
 - Loss function: Cross Entropy
 4. After completion of the training save the model to a folder. *Hint: take a look at corresponding sample code*
 5. Plot a graph for epoch vs loss for training dataset for all architecture.
 6. Evaluate the different architectures using validation data and obtain their accuracy.
 7. Choose one architecture from single hidden layer and one from two hidden layers(with best validation accuracy).
 8. Evaluate the best chosen architecture from single hidden layer and two hidden layer architectures using test data and print following metrics
 - Confusion matrix
 - Accuracy
 - Micro and macro averages of precision, recall, and F1-score.

Part B: Single and Two hidden layer Neural network architecture on the Normalized Date Fruit Dataset

Perform neural network classification and performance metric evaluation as listed below.

1. Load train, validation, and test data from original data: 'DateFruit_Train.csv', 'DateFruit_Validation.csv', 'DateFruit_Test.csv', respectively.
2. Perform z-score normalization based on the training data
 - (a) Find mean μ and standard deviation σ from the training data(per attribute).
 - (b) Apply z-score normalization on all of the three parts train, validate and test, per sample attribute value against corresponding attribute mean and standard deviation.

$$z = \frac{x - \mu}{\sigma}$$

Hint: torch tensor operations can perform matrix and vector operations instead of individually computing data points.

3. Implement a neural network for classification using PyTorch with the following specifications:
 - Input layer: 34 linear neurons (corresponding to real-valued input features).
 - All neurons in the hidden layers use the tanh activation function.
 - Hidden layers: Experiments with different numbers of hidden layers and different numbers of neurons in each hidden layer:
 - (a) Single hidden layer architectures:
 - Architecture 1: Input layer $\rightarrow$ hidden layer (34 neurons) $\rightarrow$ Output layer
 - Architecture 2: Input layer $\rightarrow$ hidden layer (64 neurons) $\rightarrow$ Output layer
 - (b) Two hidden layer architectures:
 - Architecture 1: Input layer $\rightarrow$ hidden layer (34 neurons) $\rightarrow$ hidden layer (34 neurons) $\rightarrow$ Output layer
 - Architecture 2: Input layer $\rightarrow$ hidden layer (64 neurons) $\rightarrow$ hidden layer (16 neurons) $\rightarrow$ Output layer
 - Output layer: 7 neurons (*Softmax* activation function).
4. Train the neural network using:
 - Optimizer: Stochastic Gradient Descent (SGD)
 - Loss function: Cross Entropy
5. After completion of the training save the model to a folder. *Hint: take a look at corresponding sample code*
6. Plot a graph for epoch vs loss for training dataset for all architecture.

7. Evaluate the different architectures using validation data and obtain their accuracy.
8. Choose one architecture from single hidden layer and one from two hidden layers(with best validation accuracy).
9. Evaluate the best chosen architecture from single hidden layer and two hidden layer architectures using test data and print following metrics
 - Confusion matrix
 - Accuracy
 - Micro and macro averages of precision, recall, and F1-score.

Sample NN - PyTorch Code

The following code skeleton (for 2 hidden layer NN) can be copied into a Jupyter notebook. Change the parameters/values as you see fit.

1. Write separate class for single hidden layer NN and two hidden layer NN.
2. Write training and metrics evaluation as separate functions to be reused across different dataset and architectures.

```
1
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5 from sklearn.preprocessing import LabelEncoder
6
7 # Sample for loading tensor data
8 train_orig = pd.read_csv('data/DateFruit_Train.csv')
9
10 X_train_orig = train_orig.drop(columns=['Class'])
11 y_train_orig = train_orig['Class']
12
13 X_train = torch.tensor(X_train_orig.values, dtype=torch.float32)
14     ↪ # or torch.long for integer data
15
16 # Mapping class labels to int data
17 le = LabelEncoder()
18 y_train_encoded = le.fit_transform(y_train_orig)
19 y_train = torch.tensor(y_train_encoded, dtype=torch.long)
20
21 # Manual Normalization (Z-score)
22 # Matrix X in tensor format
```

```

22 mean = X.mean(dim=0) # .mean() produces mean vector
23 std = X.std(dim=0) # .std() produces standard deviation vector
24
25
26 # Tensor addition of matrix in tensor with vector
27 # adds the vector to every row of the matrix separately
28 X = X + y # sub(-), mul(*), and div(/) operate similarly
29
30 # 1. Create a Neural Network model
31 class NeuralNet(nn.Module):
32     def __init__(self, input_size, h1, h2, output_size):
33         super().__init__()
34         self.layers = nn.Sequential(
35             nn.Linear(input_size, h1), # input layer has linear
36             ↪ neurons
37             nn.Tanh(), # activation function for hidden layer 1
38             nn.Linear(h1, h2),
39             nn.Tanh(), # activation function for hidden layer 2
40             nn.Linear(h2, output_size)
41         )
42
43     def forward(self, x): # forward step applies the model to a
44         ↪ given input and computes the various results and
45         ↪ gradients
46         return self.model(x)
47
48 # 2. Initializing model
49 model = NeuralNet(input_size=30, hidden1=128, hidden2=32,
50 ↪ output_size=6)
51
52 # 3. Initializing loss function, optimizer
53 criterion = nn.CrossEntropyLoss()
54 optimizer = optim.SGD(model.parameters(), lr=lr)
55
56 # 4. Training (one step)
57 model.train() # make sure the model is in training mode ie the
58 ↪ weights can be updated
59 outputs = model(X_train) # predict results for the training
60 ↪ data
61 loss = criterion(outputs, y_train) # compute the loss using the
62 ↪ cross entropy instance
63
64 loss.backward() # back propagation of the loss

```

```

58 optimizer.step() # updating the weights based on the optimizer
   ↪ computed gradients
59
60 train_loss = loss.item() # fetch total training loss
61
62 # 5. Prediction for validation/test data
63 model.eval()
64 with torch.no_grad():
65     outputs = model(X)
66     pred = torch.argmax(outputs, dim=1)
67     loss = criterion(pred, y).item()
68
69 # 6. Convergence criteria
70 if abs(train_loss - prev_train_loss) < threshold:
71     counter += 1
72 else:
73     counter = 0
74 prev_train_loss = train_loss
75
76 if counter >= patience:
77     print(f"\nConvergence reached at epoch {epoch}")
78     break
79
80 # Adding the model to a dictionary
81 two_models[(h1, h2)] = model
82
83 # Saving the models to the folder
84 os.makedirs("saved_models", exist_ok=True) # make sure the
   ↪ directory exists
85
86 for (h1, h2) in two_models:
87     model = two_models[(h1, h2)] # fetch model from dictionary
88     model_path = f"saved_models/model_34_{h1}_{h2}_7.pth"
89     torch.save(model.state_dict(), model_path) # save model to
   ↪ the file in saved_models directory

```

Hint: for your experiments

1. Observe that date fruit dataset attributes range in long ints, large floats and normal floats, so normalized data produce better results(i.e, using z-score or min-max normalization)
2. Choose learning rate ' lr ' of 0.01 .(values $0.01 \leq \text{lr} \leq 0.001$ considered good values for learning rate)

3. *'patience'* is the number of consecutive iteration where there is minimal change in training loss to observe convergence.
4. Choose *'patience'* value to be 10% of the number of iterations/epochs, or, minimum of 10 epochs. (the best value is about 10% for strong confidence in convergence)
5. Choose *'threshold'* value to be $1e-3$ i.e, change in training loss < 0.0001 per iteration for convergence. (<for your understanding>also observe results for $1e-4$ and $5e-4$)

Sample Code for Performance Evaluation

```
1 from sklearn.metrics import confusion_matrix, accuracy_score
2 from sklearn.metrics import precision_score, recall_score,
  ↪ f1_score
3
4 # Calculate metrics
5 conf_matrix = confusion_matrix(y_test, y_test_pred)
6 accuracy = accuracy_score(y_test, y_test_pred)
7 precision_macro = precision_score(y_test, y_test_pred,
  ↪ average='macro')
8 precision_micro = precision_score(y_test, y_test_pred,
  ↪ average='micro')
9 recall_macro = recall_score(y_test, y_test_pred,
  ↪ average='macro')
10 recall_micro = recall_score(y_test, y_test_pred,
  ↪ average='micro')
11 f1_macro = f1_score(y_test, y_test_pred, average='macro')
12 f1_micro = f1_score(y_test, y_test_pred, average='micro')
13
14 # Print results
15 print("Confusion Matrix:\n", conf_matrix)
16 print(f"Accuracy: {accuracy:.4f}")
17 print(f"Precision (Macro): {precision_macro:.4f}")
18 print(f"Precision (Micro): {precision_micro:.4f}")
19 print(f"Recall (Macro): {recall_macro:.4f}")
20 print(f"Recall (Micro): {recall_micro:.4f}")
21 print(f"F1-Score (Macro): {f1_macro:.4f}")
22 print(f"F1-Score (Micro): {f1_micro:.4f}")
```